Kubernetes PVC – PostgreSQL Persistent Storage

1. Why do we need PVC?

PostgreSQL is a **stateful application** because it stores data.

For example:

- Databases
- Tables
- Rows
- Indexes
- PostgreSQL configuration/data files

If PostgreSQL stores its data only inside the container's filesystem, the data is tied to that container.

For example:

```
PostgreSQL Container
        |
        └── /var/lib/postgresql/data
                |
                ├── databases
                ├── tables
                └── records
```

If the container is recreated, the container's writable filesystem can be lost.

Therefore, we should **not depend on the container filesystem for important database data**.

Instead, we use Kubernetes persistent storage.

```
PostgreSQL Container
        |
        | mount
        v
Persistent Storage
        |
        v
PVC
        |
        v
PV
```

```
        |
        v
Actual Storage
```

The important idea is:

```
Container = application runtime
PVC/PV    = persistent data storage
```

---

2. What happens if we don't use PVC?

Suppose PostgreSQL is running without persistent storage:

```
PostgreSQL Pod
      |
      v
Container filesystem
      |
      v
PostgreSQL data
```

If the Pod/container is recreated, the old container filesystem may disappear.

For a database, this is dangerous because:

```
Pod recreated
     |
     v
New container
     |
     v
Old database data may be gone
```

Therefore, databases such as PostgreSQL, MySQL, MongoDB, etc. normally require persistent storage.

---

3. What is a PVC?

PVC means:

```
PersistentVolumeClaim
```

A PVC is a **request for persistent storage**.

For example:

```
resources:
  requests:
    storage: 5Gi
```

means:

> Kubernetes, I need 5 GiB of persistent storage.

The PVC does not represent the actual physical disk itself. It is a request/claim for storage.

---

4. What is a PV?

PV means:

```
PersistentVolume
```

A PV represents the actual persistent storage resource available to Kubernetes.

The relationship is:

```
Application
     |
     v
    Pod
     |
     v
    PVC
     |
     v
    PV
     |
     v
Actual Storage
```

In many cases, we don't manually create the PV.

A StorageClass can automatically create a PV when a PVC requests storage.

This is called:

```
Dynamic Provisioning
```

---

5. What is StorageClass?

A StorageClass defines **how Kubernetes should provision storage**.

For example, in Minikube we saw:

```
StorageClass: standard
```

and:

```
volume.kubernetes.io/storage-provisioner:
k8s.io/minikube-hostpath
```

So the flow was:

```
PVC
 |
 | requests 5Gi
 v
StorageClass: standard
 |
 v
Minikube hostpath provisioner
 |
 v
PV
 |
 v
Minikube storage
```

On AWS EKS, a StorageClass can provision AWS EBS-backed storage instead.

---

6. PVC Namespace

A PVC is **namespace-scoped**.

If PostgreSQL is running in:

```
namespace: ecommerce
```

then the PVC used by PostgreSQL must also be in:

```
namespace: ecommerce
```

For example:

```
metadata:
  name: postgres-db
  namespace: ecommerce
```

This will NOT work if the PVC is in `default`:

```
PostgreSQL Pod
namespace: ecommerce

       X

PVC
namespace: default
```

Instead:

```
PostgreSQL Pod
namespace: ecommerce
       |
       v
PVC
namespace: ecommerce
```

Important:

```
PVC        = namespace-scoped
Pod        = namespace-scoped
StatefulSet = namespace-scoped
PV         = cluster-scoped
StorageClass = cluster-scoped
```

---

7. Our PostgreSQL setup

Our PostgreSQL application is running in:

```
namespace: ecommerce
```

We have:

```
StatefulSet
    |
    v
postgres-0
    |
    v
PVC: postgres-db
    |
    v
PV: 5Gi
```

The PostgreSQL container stores its data at:

```
/var/lib/postgresql/data
```

This is PostgreSQL's data directory.

---

8. PVC YAML

We created the following PVC:

```
apiVersion: v1
kind: PersistentVolumeClaim
```

```
metadata:
  name: postgres-db
  namespace: ecommerce

spec:
  accessModes:
    - ReadWriteOnce

  resources:
    requests:
      storage: 5Gi
```

This means:

```
PVC name       = postgres-db
Namespace      = ecommerce
Storage needed = 5Gi
Access mode    = ReadWriteOnce
```

---

9. What does ReadWriteOnce mean?

```
accessModes:
  - ReadWriteOnce
```

RWO means:

> The volume can be mounted as read-write by a single node at a time.

It does NOT necessarily mean only one Pod can ever use it.

It means the volume is attached for read/write access to one node.

For a single PostgreSQL Pod, RWO is commonly appropriate.

---

10. Check the PVC

Use:

```
kubectl get pvc -n ecommerce
```

Example:

```
NAME          STATUS   VOLUME                                     CAPACITY
ACCESS MODES
postgres-db   Bound    pvc-60a84346-f98e-45ec-abb0-7d20b217b7f5   5Gi        RWO
```

The important status is:

```
Bound
```

Bound means Kubernetes successfully associated the PVC with a PV.

---

11. PVC -> PV relationship

Our PVC:

```
postgres-db
```

was bound to a PV similar to:

```
pvc-60a84346-f98e-45ec-abb0-7d20b217b7f5
```

You can check:

```
kubectl get pvc -n ecommerce
```

and:

```
kubectl get pv
```

The relationship is:

```
PVC: postgres-db
        |
```

```
        v
PV: pvc-60a84346-f98e-45ec-abb0-7d20b217b7f5
        |
        v
5Gi storage
```

---

12. How do we mount the PVC into PostgreSQL?

There are two important sections in the StatefulSet:

```
volumeMounts:
```

and:

```
volumes:
```

They have different purposes.

---

13. volumeMounts

Example:

```
volumeMounts:
  - name: postgres-storage
    mountPath: /var/lib/postgresql/data
```

This means:

> Mount the volume named postgres-storage inside the PostgreSQL container at /var/lib/postgresql/data.

Important:

```
mountPath
```

is a path **inside the container**.

It is NOT a directory inside the PVC.

For PostgreSQL:

```
Container
     |
     └── /var/lib/postgresql/data
```

becomes the location where the persistent storage is mounted.

---

14. volumes

The volume connects the Pod to the PVC:

```
volumes:
  - name: postgres-storage
    persistentVolumeClaim:
      claimName: postgres-db
```

Here:

```
name: postgres-storage
```

is the volume name inside the Pod.

And:

```
claimName: postgres-db
```

refers to the actual PVC.

---

15. Important name relationship

These names must be understood carefully.

```
volumeMounts:
  - name: postgres-storage
```

must match:

```
volumes:
  - name: postgres-storage
```

But:

```
claimName: postgres-db
```

must match the PVC name:

```
metadata:
  name: postgres-db
```

So:

```
volumeMounts.name
        |
        | must match
        v
volumes.name
        |
        | references
        v
volumes.persistentVolumeClaim.claimName
        |
        | must match
        v
PVC.metadata.name
```

In our case:

```
volumeMounts.name = postgres-storage

volumes.name = postgres-storage

claimName = postgres-db
```

```
PVC name = postgres-db
```

---

16. Complete PostgreSQL StatefulSet

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
  namespace: ecommerce

spec:
  replicas: 1

  selector:
    matchLabels:
      db: postgres

  template:
    metadata:
      labels:
        db: postgres

    spec:
      containers:
        - name: postgres
          image: postgres:16

          ports:
            - containerPort: 5432

          env:
            - name: POSTGRES_USER
              valueFrom:
                secretKeyRef:
                  name: secret
                  key: POSTGRES_USER

            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: secret
                  key: POSTGRES_PASSWORD
```

```yaml
            - name: POSTGRES_DB
              value: ecommerce

          resources:
            requests:
              memory: "200Mi"
              cpu: "50m"

            limits:
              memory: "500Mi"
              cpu: "100m"

          volumeMounts:
            - name: postgres-storage
              mountPath: /var/lib/postgresql/data

      volumes:
        - name: postgres-storage
          persistentVolumeClaim:
            claimName: postgres-db
```

---

17. PostgreSQL Secret

The PostgreSQL container requires a username and password.

Example:

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: secret
  namespace: ecommerce

type: Opaque

stringData:
  POSTGRES_USER: admin
  POSTGRES_PASSWORD: admin
```

The StatefulSet references the Secret:

```yaml
env:
  - name: POSTGRES_USER
    valueFrom:
      secretKeyRef:
        name: secret
        key: POSTGRES_USER

  - name: POSTGRES_PASSWORD
    valueFrom:
      secretKeyRef:
        name: secret
        key: POSTGRES_PASSWORD
```

---

18. POSTGRES_DB automatically creates the database

We used:

```yaml
- name: POSTGRES_DB
  value: ecommerce
```

The official PostgreSQL container image uses this during **first-time initialization**.

Therefore, we did not manually execute:

```sql
CREATE DATABASE ecommerce;
```

The database was automatically created when PostgreSQL initialized its empty data directory.

We verified it using:

```sql
SELECT current_database();
```

which returned:

```
ecommerce
```

Important:

The initialization variables such as:

```
POSTGRES_DB
POSTGRES_USER
POSTGRES_PASSWORD
```

are primarily used when PostgreSQL initializes an empty data directory.

If the PVC already contains an initialized PostgreSQL database, changing these environment variables does not automatically recreate/reconfigure the existing database.

---

19. PostgreSQL Service

PostgreSQL normally listens on:

```
5432
```

We created a Kubernetes Service:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: dbservice
  namespace: ecommerce

spec:
  selector:
    db: postgres

  ports:
    - port: 9090
      targetPort: 5432

  type: ClusterIP
```

Here:

```
Service port = 9090
Target port  = 5432
```

Therefore:

```
dbservice:9090
       |
       v
PostgreSQL Pod:5432
```

PostgreSQL itself is still running on port:

```
5432
```

9090 is only the Service port we selected.

---

20. Why does the Service use selector db=postgres?

Our PostgreSQL Pod has:

```
labels:
  db: postgres
```

The Service has:

```
selector:
  db: postgres
```

Therefore Kubernetes finds the PostgreSQL Pod.

We verified this with:

```
kubectl describe svc dbservice -n ecommerce
```

The output showed:

```
Selector:      db=postgres
TargetPort:    5432/TCP
Endpoints:     10.244.0.5:5432
```

The endpoint proves that the Service is connected to the PostgreSQL Pod.

---

21. Complete architecture

```
                          ecommerce namespace
                                |
               +-------------------+------------------+
               |                                      |
               |                                      |
          PostgreSQL                              Service
          StatefulSet                             dbservice
               |                                      |
               v                                      |
          postgres-0                                  |
               |                                      |
               |                                      |
               +-------------> PostgreSQL <----------+
                                port 5432
                                    |
                                    v
                         /var/lib/postgresql/data
                                    |
                                    v
                                PVC
                             postgres-db
                                    |
                                    v
                                  PV
                                    |
                                    v
                          Persistent Storage
```

---

22. Complete data flow

When PostgreSQL writes data:

```
SQL INSERT
    |
    v
PostgreSQL
    |
    v
/var/lib/postgresql/data
    |
    v
volumeMount
    |
    v
postgres-storage
    |
    v
PVC: postgres-db
    |
    v
PV
    |
    v
Persistent Storage
```

---

23. What happens when the Pod is deleted?

Suppose:

```
kubectl delete pod postgres-0 -n ecommerce
```

The Pod is deleted.

But the StatefulSet still exists:

```
StatefulSet
    |
    | desired replicas = 1
    |
    v
Pod missing
    |
```

```
    v
StatefulSet creates postgres-0 again
```

The new Pod mounts the same PVC:

```
postgres-0
    |
    v
postgres-db PVC
```

Therefore the existing PostgreSQL data remains.

We tested this successfully.

Before deleting the Pod:

```
test_data
persistent-data
```

After deleting/recreating the Pod:

```
test_data
persistent-data
```

The data remained because the PVC was not deleted.

---

24. Pod deletion vs PVC deletion

This is very important.

Deleting the Pod:

```
kubectl delete pod postgres-0 -n ecommerce
```

does NOT necessarily delete the PVC.

The StatefulSet recreates the Pod and it reconnects to the existing persistent storage.

Therefore:

```
Delete Pod
    |
    v
Pod recreated
    |
    v
Same PVC
    |
    v
Data remains
```

But deleting the PVC is different:

```
kubectl delete pvc postgres-db -n ecommerce
```

Depending on the PV's reclaim policy and storage backend, the underlying storage may also be deleted.

Therefore:

```
Pod deletion  -> normally data remains
PVC deletion  -> data may be deleted
```

Never delete a production database PVC casually.

---

25. How to verify the PVC is mounted

Use:

```
kubectl describe pod postgres-0 -n ecommerce
```

Look for:

```
Mounts:
  /var/lib/postgresql/data from postgres-storage (rw)
```

And:

```
Volumes:
  postgres-storage:
    Type:       PersistentVolumeClaim
    ClaimName:  postgres-db
```

This confirms:

```
Container path
/var/lib/postgresql/data
        |
        v
Volume
postgres-storage
        |
        v
PVC
postgres-db
```

---

26. Check PVC

```
kubectl get pvc -n ecommerce
```

Example:

```
NAME          STATUS   VOLUME                                     CAPACITY
ACCESS MODES
postgres-db   Bound    pvc-xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx   5Gi        RWO
```

The important status is:

```
Bound
```

---

27. Check PV

```
kubectl get pv
```

You can see the PV associated with the PVC.

Example:

```
NAME                                        CAPACITY   ACCESS MODES   RECLAIM
POLICY   STATUS
pvc-xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx   5Gi        RWO
Delete          Bound
```

---

28. Check StorageClass

```
kubectl get storageclass
```

In Minikube, you may see:

```
standard
```

You can inspect it:

```
kubectl describe storageclass standard
```

The StorageClass determines how Kubernetes provisions the underlying storage.

---

29. PVC vs PV vs StorageClass

Remember these three concepts:

PVC

```
PVC = My application requests storage.
```

Example:

```
resources:
  requests:
    storage: 5Gi
```

PV

```
PV = Actual storage resource available to Kubernetes.
```

StorageClass

```
StorageClass = Defines how storage should be dynamically provisioned.
```

Relationship:

```
Application
     |
     v
    PVC
     |
     v
StorageClass
     |
     v
    PV
     |
     v
Actual Storage
```

---

30. Why StatefulSet is suitable for PostgreSQL

PostgreSQL is stateful.

A Deployment is commonly used for stateless applications:

```
Frontend
Backend API
Microservice
```

StatefulSet is designed for workloads that need stable identity and/or persistent storage.

For PostgreSQL:

```
StatefulSet
    |
    v
postgres-0
    |
    v
postgres-db PVC
```

The Pod has a stable identity:

```
postgres-0
```

If it is recreated, the StatefulSet maintains that identity.

---

31. Using an existing PVC vs volumeClaimTemplates

There are two approaches.

Approach 1: Existing PVC

We used this approach.

PVC:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-db
  namespace: ecommerce

spec:
  accessModes:
    - ReadWriteOnce

  resources:
```

```
    requests:
      storage: 5Gi
```

StatefulSet:

```
volumes:
  - name: postgres-storage
    persistentVolumeClaim:
      claimName: postgres-db
```

This means:

```
StatefulSet
    |
    v
Existing PVC
    |
    v
PV
```

Approach 2: volumeClaimTemplates

A StatefulSet can also automatically create PVCs using:

```
volumeClaimTemplates:
  - metadata:
      name: postgres-storage

    spec:
      accessModes:
        - ReadWriteOnce

      resources:
        requests:
          storage: 5Gi
```

Then StatefulSet creates PVCs automatically.

For example, with:

```
replicas: 1
```

you could get:

```
postgres-storage-postgres-0
```

With multiple replicas:

```
postgres-0
    |
    +-- postgres-storage-postgres-0

postgres-1
    |
    +-- postgres-storage-postgres-1

postgres-2
    |
    +-- postgres-storage-postgres-2
```

For our current setup, we already created:

```
postgres-db
```

so we used the existing PVC approach.

---

32. Testing persistence

Create test data:

```
kubectl exec -it -n ecommerce postgres-0 -- psql -U admin -d ecommerce
```

Inside PostgreSQL:

```sql
CREATE TABLE test_data (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100)
);

INSERT INTO test_data (name)
VALUES ('persistent-data');
```

```sql
SELECT * FROM test_data;
```

Then exit:

```
\q
```

Delete the Pod:

```
kubectl delete pod postgres-0 -n ecommerce
```

Watch it recreate:

```
kubectl get pods -n ecommerce -w
```

After it becomes Running:

```
kubectl exec -it -n ecommerce postgres-0 -- psql -U admin -d ecommerce
```

Then:

```sql
SELECT * FROM test_data;
```

If the data is still present, persistence is working.

---

33. Final important concepts

Remember these points:

1. PostgreSQL is stateful.
2. Do not rely on the container filesystem for important database data.
3. PVC is a request/claim for persistent storage.
4. PV represents the persistent storage resource.
5. StorageClass can dynamically create PVs.
6. PVC and Pod must be in the same namespace.
7. PV is cluster-scoped and does not have a namespace.
8. `volumeMounts.mountPath` is a directory inside the container.
9. `volumeMounts.name` must match `volumes.name`.
10. `volumes.claimName` must match the PVC name.

11. PostgreSQL's default port is `5432`.
12. `POSTGRES_DB=ecommerce` causes the official PostgreSQL image to create the database during first-time initialization.
13. Deleting the Pod does not automatically mean deleting the PVC.
14. StatefulSet recreates the PostgreSQL Pod when it is deleted.
15. The recreated Pod can mount the same PVC and access the existing database data.
16. A Service provides a stable network endpoint for PostgreSQL.
17. A Service in another namespace can be accessed using:

```
dbservice.ecommerce:9090
```

1. Our Service currently maps:

```
dbservice:9090
       |
       v
PostgreSQL Pod:5432
```

1. The Service selector:

```
selector:
  db: postgres
```

matches the PostgreSQL Pod label:

```
labels:
  db: postgres
```

1. Always be careful before deleting a PVC because the underlying persistent data may be deleted depending on the storage backend and reclaim policy.

34. Commands used in this setup

Check StatefulSet:

```
kubectl get statefulset -n ecommerce
```

Check Pod:

```
kubectl get pods -n ecommerce
```

Check PVC:

```
kubectl get pvc -n ecommerce
```

Describe PVC:

```
kubectl describe pvc postgres-db -n ecommerce
```

Check PV:

```
kubectl get pv
```

Check StorageClass:

```
kubectl get storageclass
```

Describe Pod:

```
kubectl describe pod postgres-0 -n ecommerce
```

Connect to PostgreSQL:

```
kubectl exec -it -n ecommerce postgres-0 -- psql -U admin -d ecommerce
```

Delete/restart the Pod:

```
kubectl delete pod postgres-0 -n ecommerce
```

Watch Pod recreation:

```
kubectl get pods -n ecommerce -w
```

Check Service:

```
kubectl get svc -n ecommerce
```

Describe Service:

```
kubectl describe svc dbservice -n ecommerce
```

Check Service endpoints:

```
kubectl get endpoints dbservice -n ecommerce
```

35. Final architecture

```
                         Kubernetes Cluster
                               |
              +------------------+------------------+
              |                                     |
              v                                     v
        ecommerce namespace                   Other namespace
              |                             e.g. frontend
              |
        PostgreSQL StatefulSet
              |
              v
         PostgreSQL Pod
          postgres-0
              |
              | mount
              v
    /var/lib/postgresql/data
              |
              v
     postgres-storage volume
              |
              v
        PVC: postgres-db
              |
              v
              PV
              |
              v
       Persistent Storage


Other application Pod
        |
        | dbservice.ecommerce:9090
        v
```

```
    dbservice
    ClusterIP
         |
         | targetPort: 5432
         v
    postgres-0
    PostgreSQL :5432
```

The main idea to remember is:

```
PVC protects the database data from Pod/container recreation.

volumeMounts tells Kubernetes WHERE the storage appears inside
the container.

volumes connects the Pod to the PVC.

StatefulSet manages the PostgreSQL Pod.

Service provides a stable network endpoint for PostgreSQL.
```